

Chapter 7. Data Ingestion Layer

In [Chapter 2](#), we learned about the different sources and mechanisms that generate financial data. This included public, market, alternative, and internal sources. Once data is generated at the source, its lifecycle within a financial data infrastructure begins with ingestion. As this chapter will show, data ingestion isn't as simple as the term may sound. In today's complex financial data landscape, data ingestion has expanded to encompass a large variety of data transmission and arrival processes as well as ingestion technologies, mechanisms, and formats.

Data ingestion serves as the foundational layer for information exchange in financial market operations. It facilitates communication among financial institutions and market participants for initiating transactions like payments, settlements, and trades. It also supports the exchange of inquiries and notifications between financial entities. Furthermore, compliance with regulatory requirements relies on efficient data ingestion practices, enabling financial firms and brokers to transmit various financial details and compliance reports to regulatory bodies. In addition, data ingestion is essential for disseminating market data and delivering real-time updates, critical for maintaining liquidity and operational efficiency throughout financial markets.

This chapter will examine the data ingestion layer as the primary entry point for receiving transmitted data and as a crucial bridge facilitating communication and information exchange among financial institutions and other entities.

Data Transmission and Arrival Processes

Data ingestion is a process wherein data gets transmitted from a given source, travels over a network, and arrives at its destination.

Understanding the details of such a process is critical for designing a reliable financial data infrastructure that meets different business and technical requirements. With such knowledge, financial data engineers can optimize time and mission-critical financial applications, design cost-effective and efficient data pipelines, anticipate scalability needs, manage security, and guide technology choices such as database management systems.

In this section I'll explain the standard transmission protocols that enable most data transactions worldwide and will provide an examination of the many types of data arrival patterns.

Data Transmission Protocols

When designing a system that transmits and receives data, transmission protocols, also known as communication protocols or network protocols, are an essential component. Simply put, a transmission protocol is a set of

rules, techniques, and definitions that allow two or more agents or machines to exchange data over a network such as the internet.

Understanding industry transmission protocols is crucial to mastering the art of data ingestion. A data infrastructure is reliable only if it can access, deliver, and ingest data over a network. This is particularly relevant to the financial industry, which relies substantially on data transfers due to the large volume of financial transactions.

Various communication protocols and standards are employed throughout the data transmission lifecycle, each serving specific purposes. A variety of models have been developed to establish a reference framework. Such models rely on the idea of organizing network protocols and the technologies used to implement them into distinct layers. In such a design, each protocol belongs to one layer, and each layer is interested in the services that it offers to the layer above. This defines the *service model* of the layer. In this architecture, each service performs certain actions that belong to it, uses input/instructions from the service below, and conforms to the requirements of the service above. For example, the service model of layer N might involve encrypted message delivery between systems. This could be realized by implementing nonencrypted message delivery at layer N-1 and incorporating layer N functionality to encrypt messages.

The most popular internet protocol model is the seven-layer *Open Systems Interconnection* (OSI) model, defined in standard [ISO/IEC 7498](#), and the four-layer internet protocol suite known as the *TCP/IP*, developed by the Department of Defense (DoD). Both models are illustrated in [Figure 7-1](#).

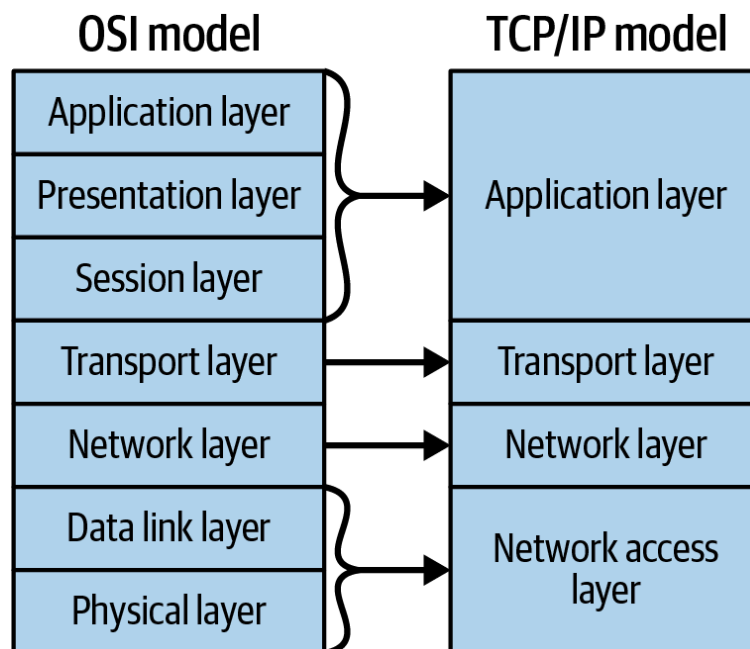


Figure 7-1. Seven-layer OSI model versus four-layer TCP/IP model

NOTE

Keep in mind that communication layer models such as OSI and TCP/IP are conceptual frameworks intended to discuss networking in a structured way. Reality may be more complex than a model. For example, some protocols or functionalities might be hard to place in one single layer and multiple layers may duplicate the same functionality. Additionally, layers might have dependencies that could blur the line between them. Nevertheless, reference models are very useful for thinking about such a complex topic.

In the following sections, I will briefly discuss the various network layers of the TCP/IP model. I chose to illustrate the TCP/IP model rather than OSI due to its simplicity and popularity among engineers.

Application layer

This is the topmost layer, where network applications and their protocols reside. Applications in this layer are distributed over multiple end systems, and they exchange packets of information via the service protocols. Such protocols include the following:

Hypertext Transfer Protocol (HTTP)

Used to serve web page requests

Simple Mail Transfer Protocol (SMTP)

Used to send and exchange emails

File Transfer Protocol (FTP)

Used to transfer files between two systems

Domain Name System (DNS)

Used to resolve web page names into their address

Secure Shell (SSH)

Used to secure login to remote servers and execute commands, or secure file transfer through the *Secure File Transfer Protocol* (SFTP)

Advanced Message Queuing Protocol (AMQP)

Used for message-oriented communication between applications

Message Queuing Telemetry Transport (MQTT)

Designed for lightweight, message-oriented communication

Transport layer

This layer is responsible for transferring application layer packets between application endpoints in a reliable, optimized, and guaranteed way. Transport layer messages are commonly called *segments*. The most important transport layer protocols include the *Transmission Control Protocol* (TCP) and *User Datagram Protocol* (UDP). TCP is a *connection-oriented* protocol, meaning that the sender and receiver must establish a continuous connection before exchanging data segments. This feature makes TCP a reliable protocol, as it acknowledges message reception and resends data again in case it doesn't arrive. TCP breaks down an information packet from the application layer into a small set of segments and

submits it for delivery to the next layer, the network layer. In the network layer, segments are represented as datagrams.

Additionally, TCP offers functionalities such as flow and congestion control, which regulate the data transmission rate and dynamically adapt to network congestion through message segmentation and breakdown. Furthermore, TCP ensures secure delivery by keeping track of all transmitted segments and confirming their successful delivery.

On the other hand, UDP is a faster but less reliable protocol than TCP. UDP is a *connectionless* protocol, meaning that data transmission happens via a fire-and-forget mechanism without establishing a continuous connection. This means that it offers fewer delivery guarantees compared to TCP.

A significant improvement over TCP is the *Transport Layer Security* (TLS) protocol, also referred to as TLS/SSL, which is designed to facilitate secure communication across insecure networks. TLS can be positioned between the application and transport layers. It encrypts data before transmission via TCP and decrypts it upon arrival at the transport layer of the receiving end. In addition to encryption, TLS also manages identity verification (authentication), facilitates the exchange of encryption keys between hosts, and mitigates Man-in-the-Middle (MITM) attacks through the [TLS handshake process](#).

TLS is very important to understand because you will encounter it several times when designing a financial data infrastructure. A common use case for TLS is enabling encrypted connections to a database engine.¹ Similarly, when using HTTPS instead of HTTP to serve web requests and design APIs, you are adding a TLS/SSL layer for encryption.

ELECTRONIC BANKING INTERNET COMMUNICATION STANDARD

Internet protocols are essential in developing financial communication protocols. A good example in this context is the *Electronic Banking Internet Communication Standard* (EBICS), developed in Germany and adopted by France, Switzerland, and other countries to exchange instructions between financial institutions and corporations over the internet. The EBICS standard has been primarily used to initiate *Single Euro Payments Area* (SEPA) exchanges over the internet, such as SEPA Direct Debits and SEPA Credit Transfers.

According to the [official technical specifications](#), the TCP/IP internet protocol suite had a decisive influence on the design of EBICS. Data is transmitted as packages via IP addresses or URLs that get resolved to IPs. Package transfer and delivery are monitored and guaranteed via the TCP protocol. The client and the EBICS server communicate using the HTTP protocol at the application layer. The XML file format has been selected as the protocol language at the application layer. To ensure security and data encryption, EBICS relies on TLS to secure communications via HTTPS.

Network layer

This layer moves information units known as *datagrams* between two hosts. When a source host wants to send data to a target host, it includes a transport layer segment along with the target host's address. Upon recep-

tion, the network layer ensures the segment is delivered to the transport layer on the target host.

The most prominent network layer protocol is the *internet protocol* (IP), with its version 4 (IPv4) being the most prevalent. IPv6 serves as its successor and has been progressively integrated into the public internet infrastructure since around 2006. To add an extra encryption layer on top of the IP protocol, the *IPsec* can be used. IPsec is commonly used with Virtual Private Networks (VPNs).

The IP protocol submits datagrams independently to the receiving host. A datagram might include several fields about its content, destination, and relevant details for the receiving end. Once received by the host, the datagrams are assembled again through the TCP protocol at the transport layer.

Understanding IP addressing is of primary importance for financial data engineers, particularly in the context of designing distributed systems such as Apache Cassandra for distributed databases or Apache Spark for distributed computing environments. In these setups, a typical strategy involves provisioning a defined number of nodes (machines) and linking them to form a cluster. Each node in the cluster is uniquely identified by an IP address, which can be used by the cluster manager and the worker nodes to establish connections and communicate with each other.

Another interesting use case of IP addressing is with Virtual Private Clouds (VPCs). One of the main concepts behind VPCs is the *subnet*, which refers to a range of IP addresses assigned to the VPC. IP addresses within a subnet are private, meaning they are inaccessible via the public internet but solely accessible through the VPC's internal network. When you provision and launch an instance of a given resource into a VPC, a primary private IP address from the subnet range is assigned to the instance.²

Network access layer

In this layer, datagrams coming from the network layer get routed through a network of routers that connect the network layer interfaces of the source and target. Datagrams are moved from one node (router) to the next until they reach their destination. Examples of network interface protocols include the Ethernet, WiFi, and Data Over Cable Service Interface Specification (DOCSIS) protocols employed in cable access networks.

The physical transmission of the individual data bits through the network happens via hardware devices that directly interface with a network, such as coaxial, optical, fiber, or twisted pair cables.

You might wonder how the physical networking layer is relevant to financial markets. Interestingly, it holds significant relevance and has been transforming certain market infrastructures, such as trading.

An illustrative example is the establishment of the transatlantic fiber-optic line [Hibernia Express](#), designed to link London and New York with ultra-low latency in response to demand from banks, exchanges, and trading firms. Thanks to Hibernia Express, London and New York experienced a five-millisecond reduction in latency compared to existing high-speed network services. In the era of automated trading, even a five-millisecond difference holds substantial importance to computers.

High-frequency traders, who have been engaged in a technical arms race to accelerate trade execution time, are [speculating on an experimental form of fiber-optic cable](#), called *hollow-core fiber*, to speed up their trades by billionths of a second.

Data Arrival Processes

Data may arrive at the ingestion layer with different temporal and structural patterns. In this section, I will use the *data arrival process* (DAP) concept to identify and describe the characteristics of a certain data ingestion pattern. A variety of DAPs may exist. Let's examine six that are particularly relevant to financial data engineering.

Scheduled data arrival process

In a scheduled DAP, data is ingested into the system according to a predetermined schedule and ingestion specifications. This process is generally more manageable as it follows a predictable pattern, with known details such as the arrival time, data type, format, volume, and ingestion method.

Scheduled DAPs are quite common in financial markets. For example, they may be used in the following circumstances:

- Arrival of company annual report filings at specific intervals such as by the end of the year, the end of March of each year, or at a future announced date
- Arrival of historical financial data snapshots from a financial data vendor on a daily, weekly, monthly, or yearly basis
- Arrival of financial institutions' required regulatory filings on a predefined date
- Arrival of data from continuous model training, stress testing, scenario analysis, and other financial quantitative analysis

Knowing when the data is expected to arrive can greatly enhance financial data engineers' jobs. This is because they can plan data ingestion jobs, anticipate the capacity and computational needs of the data infrastructure, and provision the necessary resources on a predictable basis (e.g., on a specific day of the week).

The main drawback of scheduled DAPs is the delay between data generation and arrival, which can impact data timeliness. Additionally, if scheduled jobs run without fetching any data, it can waste resources.

Event-driven data arrival process

When the arrival of data is contingent upon the occurrence of an event that cannot be predicted in advance, the DAP becomes event-driven. Event-driven DAPs are very common in financial markets. These include the arrival of the following types of data:

- Trade and quote data upon submission and execution in the market
- Financial information messages
- Transaction data upon the execution of a financial operation, such as payments, transfers, and others
- Client files for loan and credit card applications
- Data streams from financial data vendors
- News data
- Company updates and announcements
- Social media posts

The primary advantage of event-driven DAPs is that data is available shortly (or immediately) after its generation. This allows financial institutions to act promptly in response to new information and have the most recent, up-to-date market and operational insights. For this reason, event-driven systems are often associated with real-time systems. A system is considered real-time if its response time—typically set at a very low value—is essential to its proper functioning. Moreover, event-driven DAPs can save costs and optimize resource allocation as they switch resource utilization from a fixed to an on-demand pattern.

The term *real-time* is used widely in a variety of contexts. To many people, real-time would mean “instantaneously” or “immediately.” For software and system engineers, real-time may refer to time-related characteristics of a system or application, such as processing time, response time, or latency. Given the significance of real-time systems in finance, I will provide a detailed description to clear up any misconceptions. To do so, I will draw a summary from the seminal work of Seppo J. Ovaska and Philip A. Laplante, *Real-Time Systems Design and Analysis: Tools for the Practitioner* (Wiley-IEEE Press, 2011).

Authors Laplante and Ovaska define a real-time system as “a mapping of a set of inputs into a set of outputs.” The time between a system’s reception of input and the generation of the final output is called the system’s response time. The nature and speed of the response time depend on the system’s features and purpose.

Building on this, the authors define a real-time system as a “computer system that must satisfy bounded response-time constraints or risk severe consequences, including failure.” Failure, in this case, means that the system is not able to function or meet one of the requirements of its design specifications. More specifically, a failed real-time system is one that cannot satisfy timing or deadline constraints established in its specifications. In short, a system does not necessarily need to respond instantaneously to be considered real-time; it must simply define and meet response time criteria in its specifications.

At this point, however, all systems may be considered real-time as there is always some sort of time constraint. To further clarify this point, Laplante and Ovaska classify real-time systems into three categories:

Soft real-time systems

A failure to meet a response time constraint leads to performance degradation but not system failure.

Hard real-time systems

A failure to meet a single deadline can lead to complete or major system failure.

Firm real-time systems

Missing a few deadlines may not be consequential, but missing more than a few may lead to complete or major system failure.

According to business and operational considerations, real-time systems can be categorized as soft, hard, or firm. For instance, if an ATM machine occasionally fails to respond to requests within its internal time limit (e.g., 10 seconds), it might lead to some customer dissatisfaction, but it remains tolerable, thus qualifying as a soft real-time system.

Conversely, in the context of a hedge fund engaged in high-frequency trading, delays in receiving data beyond expected deadlines could result in significant financial losses, necessitating the classification of the system as hard or firm. Another example is financial systems that involve Forex currency conversions. This process frequently includes a *Request for Quote* (RFQ), where a market participant requests a price quote from a

liquidity provider or Forex broker to either buy or sell a specified amount in a particular currency pair. RFQs include an expiry time, during which the liquidity provider commits to honoring the quoted price. Failing to settle a Forex transaction within the RFQ's expiry time can expose the requesting institution to market risks, potentially resulting in financial losses.

Importantly, it is still common for people (even engineers) to think of real-time systems as being “instantaneous.” To understand why, let's take the technological concept of *real-time payments* (RTPs). According to [Stripe](#), RTPs are “instant payments that are processed immediately and continuously, 24/7.” In this context, instant refers to the fact that money is moved between bank accounts in seconds instead of hours or days (as is the case with traditional payment systems). For humans, several seconds may feel instantaneous, thus the use of the term instant in this context.

Nonetheless, I highly recommend following a systematic approach similar to the one outlined above when creating real-time financial systems. Software and data engineers must carefully understand and incorporate time constraints and limitations into their systems and investigate potential ways to change a soft real-time system into a hard or firm one, or vice versa.

With the emergence of cloud computing, event-driven data processing technologies have remarkably increased in popularity. These include message brokers (detailed in [Chapter 8](#)), such as Amazon Simple Notification Service (SNS), Amazon Managed Streaming for Apache Kafka (MSK), and Google Pub/Sub, as well as event-driven serverless computing platforms (illustrated in [Chapter 9](#)), such as AWS Lambda and Google Cloud Functions.

As event-driven DAPs are unpredictable, designing an event-driven financial data infrastructure requires careful attention. One essential feature to focus on concerns the ability of the infrastructure to scale to accommodate varying workloads and occasional spikes. Moreover, due to the data-intensive nature of event-driven DAPs, data quality issues such as errors, outliers, and timeliness may easily arise. Furthermore, event-driven DAPs may incur the issue of duplicate or concurrent ingestions, which happens when the same data or file gets ingested twice or more. This requires careful attention when designing the data infrastructure, as it might impact data consistency and potentially cause system failures.³

Homogeneous data arrival process

In a homogenous DAP, the ingested data has predetermined consistent properties. For example, if you purchase a subscription to a dataset provided by a financial data provider, you are likely to know the kind of data, the schema, the ingestion format, and other details.

A homogeneous DAP is simpler to manage and maintain, and it helps ensure data integrity and consistency. In the financial industry, a number of projects have been underway to standardize and universalize data input and exchange formats. The section [“Data Ingestion Formats”](#) will illustrate examples of standardized financial data formats.

Importantly, you should avoid overfitting your financial data architecture to handle only one type or format of data. This may cause problems if a data attribute changes or a new data type is ingested.

Heterogeneous data arrival process

In a heterogeneous DAP, ingested data may possess variable attributes such as extension, format, type, content, schema, and others.

Heterogeneous DAPs are quite common in finance. For example, financial data vendors provide their data in different formats and structures. In addition, for optimization purposes, different types of data may be stored and transmitted in specific formats (a topic that will be covered in the section [“Data Ingestion Formats”](#) in this chapter). Furthermore, different internal systems within financial institutions may generate data with their unique formats and structures.⁴

When designing for heterogeneous DAPs, the financial data infrastructure must account for various ingestible data types and possess the necessary capability to handle each. This complexity makes optimizing the infrastructure more challenging, but it also increases the financial institution’s flexibility to ingest and accommodate new data sources.

Such flexibility is critical in today’s fast-changing financial data landscape, where new data sources emerge regularly, and the amount and speed with which data is generated has grown significantly. Being able to accommodate a new data source means adding new analytical capabilities, developing new products, and gaining comprehensive insights into market trends, sales, customers, and operations.

Single-item data arrival process

In a single-item DAP, data is ingested either on a record-at-a-time or file-at-a-time basis. Think, for example, about the arrival of information related to a payment transaction, bank loan application, market order, analytical report, or piece of news.

The main advantages of single-item DAPs are traceability and transactional guarantee. When the ingestion process concerns a single data item, it is typically easier to trace its lifecycle through system logs. Moreover, inserting one record at a time allows for easier data integrity and constraint checks.

As an illustration, let’s say we have a database that stores customer financial transactions. A simple SQL-based single-item ingestion into this table would look like this:

```
-- PostgreSQL
INSERT INTO customer_transactions (
  user_id,
  transaction_id,
  time,
  transaction_type,
  amount,
  communication
) VALUES (
  195,
  'XT4h4Y453',
  '2024-01-20 10:09:42',
```

```
'1985-02-10',
'Credit Transfer',
'online course subscription fees'
)
```

In some circumstances, single-item DAPs may lead to performance bottlenecks. For example, if the number of data ingestions is remarkably high, then it may jeopardize the system's ability to handle all incoming requests. This can happen due to a max connection limit on the database side (discussed in [Chapter 8](#)) or quota limit on an API side (covered in section [“Data Ingestion Technologies”](#) in this chapter). Additionally, ingesting a large number of records one at a time can be very slow, which in turn can impact data quality dimensions such as timeliness.

Bulk data arrival process

In a bulk DAP, data is ingested in large chunks. Rather than processing one record at a time, a bulk DAP handles blocks of data or files that may contain hundreds or even millions of records simultaneously.

Bulk DAP offers performance advantages by processing large data volumes in a single request, saving overhead costs. This is ideal for tasks like bulk data loading, migration between storage systems, data archival processes, and regulatory reporting. For instance, when switching database systems, dumping data in a format compatible with the new system is far more efficient than copying records individually.

To give an example, let's consider how Snowflake's bulk loading works.⁵ Assume you have a bunch of CSV files stored in the Amazon cloud storage service AWS S3, and you are using Snowflake as a data warehouse. You wish to upload the data from your CSV files into your Snowflake table.

The first thing to do is to create a `FILE FORMAT` object in Snowflake that describes the type and format of the data to be loaded. Let's say that our CSVs use the semicolon, “;”, as a field delimiter, and we want to inform Snowflake to skip the first line in each file as it represents the header. The following command creates the desired format object:

```
-- Snowflake SQL
CREATE OR REPLACE FILE FORMAT s3csvformat
  TYPE = 'CSV'
  FIELD_DELIMITER = ';'
  SKIP_HEADER = 1;
```

Next, we want to create the so-called stage object, which tells Snowflake the location where the files are stored (staged). Snowflake provides several [types of stage objects](#), but in this example, we will use the recommended Named stage,⁶ which may be created as follows:

```
-- Snowflake SQL
CREATE OR REPLACE STAGE s3_csv_stage
  FILE_FORMAT = s3csvformat
  URL = 's3://snowflake-docs';
```

Finally, to load the data from the stage location into the Snowflake table, we can execute the following command:

```
-- Snowflake SQL
COPY INTO destination_table
  FROM @s3_csv_stage/myfiles/
  PATTERN='.*daily_prices.csv'
  ON_ERROR = 'skip_file';
```

The `PATTERN` clause specifies that the command should load data from any file that matches the specified regular expression `.*daily_prices.csv`, which matches any file that ends with `daily_prices.csv`. Furthermore, the command specifies that if an error occurs when loading a specific file, skip it and proceed with the remaining files.

Now that we've covered the various data transmission and arrival processes, let's explore the different types of data formats that can be ingested into a financial data infrastructure.

Data Ingestion Formats

Data can be ingested in a variety of formats. A data format is used to indicate the extension or encoding used to store the data on a machine. In general, there is no standardized classification of the many data formats that a financial data infrastructure may support. To establish a baseline, this section will illustrate the most common types of data formats that financial data engineers might encounter when working with financial data.

General-Purpose Formats

General-purpose formats are widely used data formats with broad applicability and extensive adoption within the financial markets. Examples include the following:

- Comma-separated values (CSV) files are text files with comma-separated values (,).
- Tab-separated values (TSV) files are text files with tab-separated values (\t).
- Text files (TXT) are text files with lines delimited by a line separator (\n).
- JavaScript Object Notation (JSON) files are structured as a collection of name/value pairs.
- Extensible Markup Language (XML) files structure and store data in a hierarchical format using custom tags.
- Microsoft Excel files work with [Microsoft Excel \(e.g., XLSX and XLS\)](#).
- Compressed files are compressed using a compression algorithm such as GZip or Zip.

Financial markets use these formats for a variety of reasons and purposes. For example, Microsoft Excel files are [quite popular](#) among financial professionals and accountants due to their reliability (Microsoft as maintainer), simplicity, and advanced analytical capabilities. CSV and TSV formats are widely used for storing and sharing financial time series and tabular data. TXT files are used to store textual financial content such as reports, news, entity and reference data, and many more. JSON and XML are widely used for programmatic and web-based financial data ex-

change due to their user-friendly nature and dependable technical specifications. Consequently, they frequently form the basis for financial data standards.⁷

The downside of using general-purpose formats is that their flexibility increases the chance of data errors or quality issues. Moreover, these formats may not be efficient when dealing with large data volumes. To address this concern, more specialized formats can be employed, which will be discussed in the following sections.

Big Data Formats

General-purpose formats such as CSV and XML can easily encounter performance issues when ingesting and exchanging large amounts of data. To solve this issue, several big data formats have been developed, including Apache Parquet, Apache Avro, and ORC.

Apache Parquet is an open source, column-oriented data file format that supports efficient and economical data storage and retrieval. Parquet offers efficient data compression, decompression, schema evolution, and encoding algorithms to handle complex and large data. Parquet is accessible in several languages, including Python, C++, and Java. Parquet files are widely used. For example, the leading cloud data warehouse provider, Snowflake, [reports that Parquet is the file format most often used by its customers](#) to upload data to the Snowflake platform.

COLUMN-ORIENTED VERSUS ROW-ORIENTED FILE FORMATS

When dealing with file formats, it's important to understand the difference between column-oriented and row-oriented formats. In row-oriented formats (e.g., PostgreSQL's internal data format), data is stored on disk row by row. These formats are preferable for small datasets, strict data consistency requirements, or applications with heavy write/update operations, such as financial systems handling transactions like payments and clearing.

In contrast, column-oriented formats like Parquet store data on disk column by column. This format is highly advantageous for read-intensive and big data applications. Queries are more efficient and economical because read-intensive applications often retrieve only a subset of columns, avoiding unnecessary querying of other columns. Additionally, column-wise storage enhances compression efficiency as it optimally compresses data of the same type within a single column, unlike row-oriented formats, which compress heterogeneous data within a single row.

Apache Avro is another common big data format known for its row-oriented structure and compact binary encoding, which helps reduce file storage size. Avro stores data definitions, types, and protocols in an easily readable JSON format, while the actual data is stored in a highly optimized binary format. Avro is schema dependent, meaning that the data and its schema are stored and transmitted together in the same file. As a result, Avro is preferred over Parquet when frequent schema changes occur, as merging schemas from multiple files can be quite costly.⁸

A third common big data format is Optimized Row Columnar (ORC), a column-oriented, binary format primarily used for storing Hive data in Hadoop environments. ORC is [renowned for its exceptional performance](#) in terms of data processing speed and storage efficiency, making it well-suited for handling large volumes of data.

In-Memory Formats

In many applications, data is frequently read and processed in memory. Crucially, different software programs may store data in memory using different formats. If data moves from one application to another during a data processing pipeline, then each application needs to convert the data to its in-memory format before processing it. This is a costly operation as it often involves data serialization and deserialization, which in turn can impact performance.

To solve this issue, a variety of in-memory data formats have been developed. A prominent example is [Apache Arrow](#), a standardized, column-oriented, language-agnostic data format for structuring and representing tabular datasets in memory. Apache Arrow can be used to develop a data infrastructure that processes data across multiple systems using an out-of-the-box standardized format.

Another noteworthy example is the [Resilient Distributed Dataset \(RDD\) abstraction](#) created for Apache Spark to enable reliable, fault-tolerant, and parallel computations in memory.

Standardized Financial Formats

Market participants can exchange financial information in any of the formats covered thus far, such as CSV, JSON, TXT, and XML. However, if each financial institution employs its own convention to structure its financial messages and communications, markets will incur significant costs in understanding and extracting information from each and every format. For instance, imagine a network of one thousand trading firms where each firm submits trade request messages using its own JSON or XML structure. This scenario would result in hundreds of different message formats that every trader would need to understand.

To address this challenge, financial market participants have been working on creating standardized formats for financial information. This effort is often described by industry experts as establishing a “common financial language” for data exchange.⁹ Several initiatives have been proposed, leading to the development and adoption of multiple standards. Examples include the following:

- Financial products Markup Language (FpML)
- Financial Information eXchange (FIX)
- Interactive Financial eXchange (IFX)
- Market Data Definition Language (MDDL)
- Financial Electronic Data Interchange (FEDI)
- Open Financial Exchange (OFX)
- eXtensible Business Reporting Language (XBRL)
- Financial transaction card-originated messages (ISO 8583)
- Securities—Scheme for messages (ISO 15022)
- Universal Financial Industry Message Scheme (ISO 20022)

- SWIFT proprietary messages

In the following sections, we will go over some of these financial data standards in depth.

Financial Information eXchange (FIX)

[Financial Information eXchange \(FIX\)](#) is an electronic communication protocol widely used to exchange financial transaction information between financial institutions such as banks, trading firms, brokers/dealers, security exchanges, and even regulators. FIX is a nonproprietary open standard owned and maintained by the FIX Trading Community member firms. FIX was originally developed to exchange pre-trade and trade equities trading messages. Over time, its scope has expanded to include support for post-trade activities, as well as transactions in fixed income, foreign exchange, and listed derivatives markets.

A full account of the [technical specifications of the FIX infrastructure](#) is beyond the scope of this section. However, the FIX system can be broken down into (1) a standardized message format, (2) a FIX order routing network, and (3) a FIX engine necessary to submit and receive FIX messages. The classical encoding of FIX messages is called *tagvalue encoding*, which structures a message as a chain of tag/value pairs. Tags are integers that identify the field, followed by the “=” character (hexadecimal 0x3D), and finally, the value of that field, encoded in the ISO 8859-1 character set. Each tag/value pair is separated by the ISO 6429:1992 Start of Heading control character <SOH> (hexadecimal value 0x01). Other encodings include FIXML, which leverages XML and JSON to format the message.

For example, a single buy order FIX message looks like this:¹⁰

```
8=FIX.4.2^A 9=145^A 35=D^A 34=4^A 49=ABC_DEFG01^A 52=20090323-15:40:29^A 56=CCG^A 115=XYZ^A 11=NF 0542/0
```

The fields can be broken down as follows:

```
8=FIX.4.2 FIX version number
9=145 Body Length: 145 bytes
35=D Message Type: New Order - Single
34=4 Message Sequence Number: 4
49=ABC_DEFG01 Sender Company ID: ABC_DEFG01
52=20090323-15:40:29 Sending Time: March 23, 2009, 15:40:29
56=CCG Target Company ID: CCG
115=XYZ On Behalf Of Company ID: XYZ
11=NF 0542/03232009 Client Order ID: NF 0542/03232009
54=1 Side: Buy
38=100 Order Quantity: 100 shares
55=CVS Symbol: CVS
40=1 Order Type: Market
59=0 Time In Force: Day
47=A Special Instructions: Agency single order
60=20090323-15:40:29 Transaction Time: March 23, 2009, 15:40:29
21=1 Handling Instructions: Automated execution
207=N Security Exchange: NASD OTC
10=139 Checksum: 139
```

According to the FIX protocol, to exchange FIX messages between two financial institutions, both must have a FIX engine that communicates over a FIX routing network. Several FIX network routing options are available,

including the internet, leased lines, point-to-point VPNs, and Hub-and-Spoke. The FIX engine needs to be implemented and connected to the selected routing network to exchange messages.¹¹

eXtensible Business Reporting Language (XBRL)

eXtensible Business Reporting Language (XBRL) is an XML-based open international standard for digital business and financial reporting. XBRL is managed and maintained by XBRL International, a nonprofit consortium. In a nutshell, XBRL provides a standardized, accurate, and reliable way to represent and exchange business and accounting data in both human-readable and machine-readable formats. XBRL is widely used by regulators, companies, governments, data providers, investors, analysts, and accountants.

The main building block of XBRL is an XBRL *instance*, which refers to the collection of business facts that an XBRL document contains. More technically, an XBRL is an XML file whose root element is <xrli:xbml>.

Another important element is XBRL *facts*, which represent individual pieces of information in an XBRL instance. For example, a fact may say that Heckler & Brothers Inc.'s 2018 revenues were \$5 billion. The fact is reported as a value of 5b against a corresponding concept representing "Revenues," in addition to associated contextual information for the units (dollars), the period (2018), and the entity ("Heckler & Brothers Inc."). Technically, facts are represented by elements in an XBRL document.

Additionally, there are XBRL *concepts*, which can be used to describe the meaning of facts. For instance, "Assets," "Liabilities," and "Net Income" are examples of these concepts. Technically, concepts are represented as element definitions in an XML schema.

Finally, XBRL *taxonomies* correspond to collections of concept definitions. Taxonomies are typically created to represent a given reporting regime, such as international financial reporting standards (IFRS) and generally accepted accounting principles (GAAP) standards, as well as for reporting requirements of various regulators and government agencies. A taxonomy is used to define what needs to be reported clearly. At a technical level, a taxonomy is an XML schema document containing element definitions and a collection of XML documents with additional information associated with concept definitions.

XBRL allows for defining business rules that can constrain and verify what kind of data can be reported. Rules can be logical or mathematical, and they are often used to control the data quality of XBRL documents.¹²

Financial products Markup Language (FpML)

Financial products Markup Language (FpML) is an open source, XML-based information exchange standard designed for the electronic trading and processing of financial derivatives instruments. The standard is defined and maintained by the FpML Standards Committee.

Among the most distinguishing aspects of derivative markets is the flexibility in defining and shaping derivative contracts to meet specific client requirements. Moreover, a large portion of derivative trading happens

over the counter (OTC), meaning that such transactions are conducted business-to-business and not through a centralized trading venue.

With such flexibility comes a large variety of data communication and representation styles. On the one hand, this has been considered necessary as it allows two parties to customize a derivative product to meet specific client needs. Consequently, attempts to standardize OTC derivative communications have not gained much traction, as they were doomed to become obsolete quite fast once new requirements emerged. This situation has led to a manual data exchange process between trading parties, which is prone to errors.

However, with the increase in derivative trading volume and the establishment of new requirements for derivative trade processing, standardization became more appealing. In this regard, FpML was introduced to automate the flow of information across the entire derivative trading network (of partners and clients), independent of the underlying software or hardware infrastructure supporting the related activities. The standard was initially developed for interest rate derivatives such as swaps, but it has been extended to other classes of derivatives, structured products, bonds, and commercial loans since then. Moreover, FpML has evolved to cover the different stages of a derivative transaction, such as pre-trade, trade, and post-trade.

An FpML message is encoded using Unicode Transformation Format (UTF)-8 or UTF-16 and uses XML as the file format. FpML may include a number of elements whose values are restricted to a limited set of values (e.g., currencies). Such restricted sets are called *domains*. FpML relies on two types of domain codings. First are domains that don't change frequently throughout the life of the specification, which are coded using XML schema enumerations. The second type is domains coded using a strategy defined by the Architecture Working Group, referred to as *schemes*. A scheme is associated with a Uniform Resource Identifier (URI). Three categories of coding schemes exist:

- External coding scheme with a URI assigned by an external body such as an open standards organization or a market participant
- External coding scheme without a URI; in such a case, FpML assigns a URI
- An FpML-defined coding scheme, defined and versioned by FpML, which also assigns the URI

For example, an FpML-defined scheme is *actionTypeScheme*, which codes the action types as defined by the European Securities and Markets Authority (ESMA).¹³ As of the time this book was written, the URI for this scheme is <http://www.fpml.org/coding-scheme/action-type-1-0>, and its coding scheme is shown in [Table 7-1](#).

Table 7-1. FpML-defined action type scheme

Code	Description
C	Cancel (a termination of an existing contract)
E	Error (a cancellation of a wrongly submitted report)
M	Modify (a modification of details of a previously reported derivative contract)
N	New (a derivative contract reported for the first time)
O	Other (any other amendment to the report)
V	Valuation update (an update of a contract valuation)
Z	Compression (a compression of the reported contract)
C	Cancel (a termination of an existing contract)

FpML's flexibility, enabled by its use of schemes, makes it well-suited for handling custom exchange definitions and requirements in the derivatives markets. Those interested in exploring the specifics of these schemes can refer to the [comprehensive FpML Coding Schemes documentation](#).

Open Financial Exchange (OFX)

[Open Financial Exchange \(OFX\)](#) is a widely adopted open standard for the electronic exchange of financial data and instructions between financial institutions, businesses, and customers. OFX allows direct connection between customers and institutions without requiring an intermediary.

Open Financial Exchange relies on open specifications that anyone can implement (e.g., financial institution, software development firm, transaction processor, or other party). A client-server model is used to design the OFX system. The client submits a request (e.g., HTTP) to an OFX server, and the server replies with a response to the client. OFX defines the request/response message structure and provides guidelines for building the infrastructure for supporting message exchange. OFX uses widely accepted open standards for networking (such as TCP/IP and HTTP), data formatting (such as XML), and TLS.

OFX is widely utilized by financial institutions for a variety of applications and use cases. Since 1997, it has been the [leading direct API standard](#) for banks to provide data to financial applications. It is currently in use at over 7,000 financial institutions. OFX can be implemented and adapted across a wide range of frontend applications and platforms. Moreover, it is an extensible standard, allowing for the straightforward addition of new services as needed.

Readers are encouraged to check the [official documentation](#) for more details on OFX's technical implementation and message structure.

Universal Financial Industry Message Scheme (ISO 20022)

As messaging standards increased in scale, sophistication, and variety, financial market participants, in collaboration with the ISO, initiated discussions to make uniform the message standardization process. This resulted in the introduction of the highly celebrated [ISO 20022—Universal Financial Industry Message Scheme](#).

ISO 20022 is an open and global standard that aims to streamline financial market communication and messaging using a common language. Its general-purpose design makes it suitable for the majority of use cases, irrespective of the business domain, communication network, or counterparty.

A distinguishing feature of ISO 20022 is its model-based approach. When you use ISO 20022 to develop a new message, the outcome is a model that defines and describes all parts of the message exchange and communication protocol between participants. The modeling method consists of four levels: scope, conceptual, logical, and physical. These levels are developed as one progresses from the business process and its associated features and components to the development of the final instance of the message model in a given syntax (typically XML).

Having said this, it is important to remember that ISO 20022 is not a single communication standard in and of itself but rather a standard that describes a development methodology for creating financial message models.

ISO 20022 models and their related message components are available online via a central [repository](#) organized into two areas:

The Data Dictionary

Contains industry model elements such as business concepts, message concepts, and data types. These elements are called dictionary items and serve as reusable components for future models.

The Business Process Catalog

Contains model message definitions and syntax implementations.

According to the [official ISO documentation](#), the modeling methodology of ISO 20022 consists of eight parts:

ISO 20022-1

Metamodel for all models and the repository

ISO 20022-2

UML profile to create models that conform to the ISO 20022-1 Metamodel

ISO 20022-3

Modeling method to produce models

ISO 20022-4

XML schema generation rules to transform a logical-level model into a physical-level implementation

ISO 20022-5

Reverse engineering guidelines to extract relevant information from existing messages

ISO 20022-6

Message transport characteristics

Abstract Syntax Notation One (ASN.1) generation rules to transform a logical model into an ASN.1-based physical level

ISO 20022 messages follow a four-block convention for naming. For instance, a well-known message is the “FinancialInstitutionToFinancialInstitutionCustomerCreditTransfer” message, represented in the ISO 20022 convention, as shown in [Figure 7-2](#). In this example, “PACS” denotes “Payment Clearing and Settlement,” indicating the message’s business domain of payment and settlement instructions. The “008” segment serves as the message type identifier, specifying the type of transaction—in this case, a financial institution to financial institution customer credit transfer. The “001” designation represents the variant number, indicating the global message definition. Lastly, “12” identifies the message version within the ISO system.

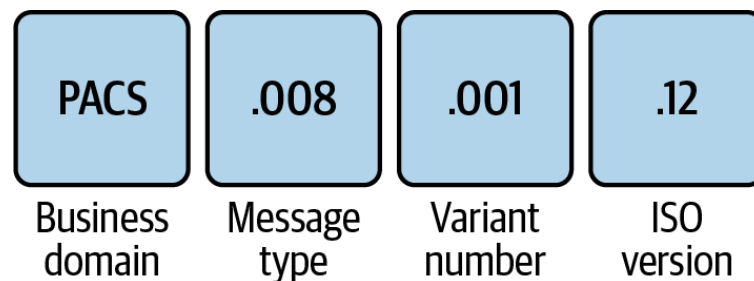


Figure 7-2. Structure of an ISO 20022 message

ISO 20022 VARIANTS

[Variants](#) and versions are pivotal aspects of ISO 20022, allowing for the creation of simplified and purpose-specific versions of the global message to suit specific requirements.

One of ISO 20022’s nicest features is the concept of a variant. Each ISO 20022 message has a *global message definition*, typically represented with variant number 001. Other variants (>001) can be created to produce restricted versions of a global message definition. For example, a straight-through processing (STP) variant of a global message definition may exclude all the options that would require manual processing of the message instance, and thus ensures the STP of the messages.

Each variant can have multiple versions, which is the last component you see in the message identifier shown in [Figure 7-2](#). Versions are independent of the variant; for example, variant 001 can have versions 001.001 and 001.002, while variant 002 may have versions 002.001, 002.002, and 002.003.

ISO 20022 variants play a crucial role by enabling customization of global message definitions to align with specific operational and processing needs in financial transactions. In addition, they simplify the adoption of ISO 20022 message definitions by reducing complexity and providing clarity on how to apply message definitions in specific contexts.

Examples of some of the most used ISO 20022 messages include the following:

pain.001—Credit Transfer

Customer-initiated credit transfers to banks

pain.013—Request to Pay

Requests payment from a payer

pain.002—Payment Status Report

Status updates on initiated payments

pacs.008—FI to FI Customer Credit Transfer

Customer credit transfers between banks

pacs.003—FI to FI Customer Direct Debit

Direct debit transactions between banks

pacs.002—FI to FI Payment Status Report

Status updates on financial institution payments

camt.05x

Various account reporting messages (e.g., *camt.054—Bank to Customer Debit/Credit* can be used to notify an account holder of debit and/or credit entries reported to their account)

Here is a simple XML snippet that illustrates a PACS.008.001 message. Each tag within the message has been annotated with a comment detailing its meaning or purpose:¹⁴

```
<FIToFICstmrCdtTrf> Financial Institution Credit Transfer
  <GrpHdr> Group Header
    <MsgId>123456789</MsgId> Message Identification
    <CreDtTm>2022-05-20T14:30:00</CreDtTm> Creation Date and Time
    <NbOfTx>1</NbOfTx> Number of Transactions
    <CtrlSum>1000.00</CtrlSum> Control Sum (Total Amount)
  </GrpHdr>
  <CdtTrfTxInf> Credit Transfer Transaction Information
    <PmtId> Payment Identification
      <EndToEndId>00001</EndToEndId> End-to-End Identification
    </PmtId>
    <Amt> Amount
      <InstdAmt Ccy="USD">1000.00</InstdAmt> Instructed Amount
    </Amt>
    <Cdtr> Creditor
      <Nm>John Smith</Nm> Name of the Creditor
    </Cdtr>
    <CdtrAcct> Creditor Account
      <Id> Identification
        <IBAN>GB29NWBK60161331926819</IBAN> IBAN Number
      </Id>
    </CdtrAcct>
    <RmtInf> Remittance Information
      <Ustrd>Invoice payment for services rendered.</Ustrd>
    </RmtInf>
  </CdtTrfTxInf>
</FIToFICstmrCdtTrf>
```

Any community user or organization can use the ISO 20022 modeling methodology to develop and submit a proposal for a new model or a modification of an existing model. Candidate models are reviewed and approved by three registration bodies: the Registration Management Group (RMG), the Registration Authority (RA), and the Standards Evaluation Groups (SEGs).¹⁵

ISO 20022 has seen remarkable adoption and acceptance among market participants. This includes all domains where financial data is exchanged, including payments, securities trading and settlement, credit and debit card transactions, foreign exchange transactions, and many more.¹⁶ For example, SWIFT introduced ISO 20022 in March 2023 and [established a migration plan](#) in which both SWIFT proprietary MT (message type/text) messages and ISO 20022 will coexist until November 2025. After that, SWIFT messages will be completely based on ISO 20022.

A good example demonstrating the use of messages in financial markets is the Society for Worldwide Interbank Financial Telecommunication (SWIFT). SWIFT is a Belgium-based cooperative that provides a secure and reliable messaging system for financial transactions worldwide. SWIFT does not hold or manage financial assets. Instead, it offers a platform for exchanging financial messages, such as money and securities transfer instructions. Over 11,000 financial institutions globally are connected to the SWIFT system.

SWIFT provides different messaging formats and schemas. The most common is the FIN message, which follows a store-and-forward mode: messages sent from the source are stored at a central intermediary location before being transmitted to the recipient. Another format is InterAct, which is XML-based and offers features such as real-time messaging and query-and-response capabilities. Finally, FileAct messages are used to transfer files, such as large batches of messages or other payment-related files. Many of SWIFT's messaging formats are based on ISO standards, such as ISO 15022 and, more recently, ISO 20022.

SWIFT messages are categorized using the [convention MT \(message type/text\)](#), followed by three digits that indicate the message category, group, and type (e.g., MTxxx). There are nine message categories in the SWIFT system; for example, MT1xx is for customer payments and checks, and MT5xx is for securities market transactions.

Consider a scenario where Corporation A wants to send \$500,000 to Corporation B. Corporation A initiates the transaction by submitting payment instructions to its bank (Bank 1) using an MT101 message (or ISO 20022 pain.001). Upon receiving the message, Bank 1 issues a credit transfer request to Corporation B's bank (Bank 2) using an MT103 message (or ISO 20022 pacs.008). [Figure 7-3](#) illustrates the flow.

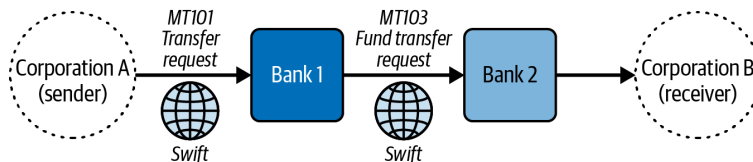


Figure 7-3. A SWIFT transfer

In this example, it was assumed that Bank 1 and Bank 2 have a corresponding banking relationship, which allows them to directly exchange messages such as MT101 and MT103. However, if they do not have a corresponding banking [relationship](#), the payment process would involve one or more intermediary or correspondent banks. Recently, SWIFT launched SWIFT GPI (Global Payments Innovation) to enhance the cross-border payment experience, addressing the industry's demands for greater speed, traceability, and transparency.

Now that you have an understanding of data ingestion processes and formats, let's explore the various technological options for integrating a data ingestion mechanism into a financial data infrastructure.

Data Ingestion Technologies

To enable and integrate data ingestion capabilities into a financial data infrastructure, one or more ingestion mechanisms need to be implemented. To this end, a number of technological options are available to meet varying business and technical needs. In this section, I will discuss six of the most common ingestion technologies used in financial markets.

Financial APIs

API stands for *application programming interface*, and it refers to a wide range of software implementations that allow one software component to interact with another. An API defines the rules, protocols, and methods for interacting between two software types.

At the highest level, an API can be described in terms of a client and a server. A client using one application sends a request to a server operating in another application. The server processes the request and returns a response to the client. For instance, when you open a social media application to upload a new image, you (the client) interact with the app provider's server via an API.

Technically, there are several approaches to designing and implementing APIs. These include SOAP APIs, RPC APIs, GraphQL, WebSocket APIs, and REST APIs. Among these, REST APIs, which stands for Representational State Transfer, are the most popular. REST APIs primarily use the HTTP protocol for communication. They can implement various HTTP request methods such as GET (to retrieve data, e.g., account balance or transaction history), POST (to initiate a backend process), and PUT (to create or update a resource on the backend).

NOTE

Although both PUT and POST can be used to create or update a resource, the main difference is that PUT is *idempotent*. This means that making the same PUT request multiple times will produce the same result each time. For example, if you submit a request through your banking application to pay an electricity bill, submitting the same request again should not result in the bill being paid twice.

APIs can be written in most programming languages, including Java, C, C++, Node.js, and Python. Java and Python APIs are quite common among data engineers. Within the Python ecosystem, frameworks such as Flask, FastAPI, and Django are widely used to program web-based APIs. To provide a short example, consider the following statement:

```
# Bash
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"idType": "TICKER", "idValue": "AAPL", "exchCode": "UN"}' \
  https://api.openfigi.com/v2/mapping
```

This `curl` command is making a POST request to the OpenFIGI API (`https://api.openfigi.com/v2/mapping`) to fetch mapping information for the financial instrument with the ticker symbol AAPL (Apple Inc.) on the exchange with code UN (New York Stock Exchange). To test this API

request, open a terminal or command line window on your computer, paste the command, and press Enter. The response is expected to have a structure similar to this:

```
[
  {
    "data": [
      {
        "figi": "BBG000B9XVV8",
        "name": "APPLE INC",
        "ticker": "AAPL",
        "exchCode": "UN",
        "compositeFIGI": "BBG000B9XRY4",
        "uniqueID": null,
        "securityType": "Common Stock",
        "marketSector": "Equity",
        "shareClassFIGI": "BBG001S5N8V8",
        "uniqueIDFutOpt": null,
        "securityType2": "Common Stock",
        "securityDescription": "AAPL"
      }
    ]
  }
]
```

APIs are everywhere. In finance, they are extensively used for all kinds of purposes. Payment APIs, often known as payment gateway APIs, are a major application field. In simple terms, a payment gateway is a technology that facilitates the acceptance and processing of electronic payments between merchants and financial institutions. This includes credit and debit card payments, digital wallets, and bank transfers. At the core of a payment gateway technology is an API that handles all the payment life-cycle phases and allows the involved entities (merchant, processor, gateway, financial institution, etc.) to talk to each other. Among the most known payment gateway APIs are Square, Stripe, PayPal, Authorize.Net, and Adyen.

APIs play a pivotal role in accessing data from financial data vendors. These APIs are designed to allow clients to retrieve and import data programmatically, enabling the development of data-driven financial applications. For instance, Bloomberg offers the [Server API \(SAPI\)](#) to allow customers to access Bloomberg Terminal data through both proprietary and third-party applications. Similarly, LSEG (formerly Refinitiv) provides the [Eikon Data API](#), a Python-based library that enables users to access Eikon data using Python.

TIP

When ingesting financial data from external sources, check if the data provider has an API in place. Using the API can be quite convenient and accelerate development. At the same time, make sure you check any vendor-specific API limitations that might impact your application, such as a single-request size limit (e.g., 100 prices per request), request rate limits (e.g., 1K requests/day), request timeout, and maximum concurrent requests. If you'd like to learn more, check the FactSet's Formula API limitations available on the vendor's official [web page](#).

Recently, there has been [growing interest](#) among financial market participants in open finance initiatives. These initiatives seek to establish a digital ecosystem that enables seamless sharing of financial data between financial institutions and third-party service providers. The primary driver of this new paradigm is to foster collaboration among market participants to produce better goods and services driven by financial data.

The most popular example of an open finance initiative is commonly referred to as [open banking](#), in which traditional banking institutions and third-party service providers such as FinTech firms collaborate to provide innovative financial products. To facilitate collaboration, banks and FinTechs share data using ad hoc financial APIs.

Various regulatory frameworks have been developed to promote open finance. For example, the [second Payment Services Directive \(PSD2\)](#), adopted in 2015 by the EU, imposed an obligation on banks to facilitate access to payment data for third-party service providers via a secure interface. To promote a wider open finance ecosystem that goes beyond payment account data, the EU proposed the [Financial Data Access legislation](#), whose main goal is “to establish a framework governing access to and use of customer data in finance.”

Special types of open banking enablers, regulated under frameworks such as PSD2 in Europe, include *Account Information Service Providers* (AISPs) and *Payment Initiation Service Providers* (PISPs). An AISP, or a company with an AIS license, collects, aggregates, and facilitates access to a user’s financial information across accounts held with various institutions. PISPs, on the other hand, facilitate direct payments from the consumer’s bank account to online merchants.

Various enablers have emerged to offer platforms for open banking. For instance, Tink, a Swedish company later acquired by Visa, facilitates connections to over 6,000 banks throughout Europe. Another example is Powens, a French company that connects over 1,800 institutions across numerous European countries, providing an open finance platform to its clients.

Crucially, it is important to keep in mind that financial APIs are not simply a bunch of FastAPI or Flask methods. Instead, APIs should be designed with business, user, and application requirements in mind. For example, JPMorgan Chase [classifies financial APIs into data and service APIs](#). Data APIs are mainly used to request financial data, but they can be designed in a way that they can be essential tools for building applications, enable external collaborations, and be reusable across multiple departments, channels, and product lines. Service APIs, on the other hand, are used to create and trigger an instance of a service, such as initiating a payment, a balance inquiry, or requesting a change from the bank. Many firms use the term *API integration* to denote a strategy where applications are connected via their APIs. The goal is to create an infrastructure in which data exchange and communication occur seamlessly through APIs, facilitating creativity and innovation.

When designing a financial API, the two most important elements to consider are performance and security. API performance is often measured

by the ability of the system to scale to a large number of concurrent requests, as well as its request response time. Measures such as “hits per sec” and “requests per sec” are often used for this purpose. The idea is to measure the ability of an API to handle a large number of requests/hits in one second. Common API performance optimization techniques involve load balancing, caching, rate limiting, and throttling, among others.¹⁷

As for security, the primary elements to consider are authentication and authorization to control how and who can interact with the API. Tools such as firewalls, authentication tokens, OAuth 2.0, API keys, and API gateways are often used for this purpose.¹⁸ In addition, APIs need to be protected against advanced malicious attacks such as SQL injection (SQLi). In SQLi, a cybercriminal exploits application vulnerabilities to inject malicious input into an API request that alters the behavior of a back-end SQL query. For example, let’s take a naive scenario where the user is required to insert their user ID to access their account balance. Upon providing such credential (e.g., user ID: 267), an API request is sent to the backend, which then executes the following SQL query:

```
-- SQL
SELECT first_name, last_name, account_balance
FROM user_accounts
WHERE user_id = 267
```

Now, if the API doesn’t handle SQL injection properly, then it is possible to provide input such as user ID “267 OR 1=1”. In this case, your backend may end up executing the following query:

```
-- SQL
SELECT first_name, last_name, account_balance
FROM user_accounts
WHERE user_id = 267 OR 1=1
```

Because the SQL condition `1=1` always evaluates to `TRUE`, the entire `WHERE` statement will be true, regardless of whether the provided user ID is correct. In this case, the entire list of users and their account balances will be queried, which may lead to major data breaches.¹⁹

Financial Data Feeds

In the financial industry, the term “data feed” refers to a mechanism designed to deliver the latest financial data and updates to traders, investment firms, and financial institutions. The main sources of data feeds include stock exchanges, financial news providers, and market data vendors.

Financial data feeds can be designed to transmit data either as historical snapshots or, more commonly, in live mode. When provided in live mode, a data feed offers a continuous stream of real-time data, essential in scenarios where timely data access is critical. Moreover, data feeds often include features that allow users to configure the timing, location, and specific data they want to receive. Financial data feeds may vary in terms of latency, throughput, and delivery guarantees.

Examples of financial data feeds include S&P Global’s [Xpressfeed](#), which offers access to over 200 datasets and allows users to customize data ex-

traction and delivery locations. LSEG's [Real-Time – Ultra Direct](#) is another example, providing a high-performance, low-latency real-time market data feed. In addition, Bloomberg provides the [Bloomberg Market Data Feed \(B-PIPE\)](#). Exchange venues also offer market data feeds, such as the NYSE [Trades Data Feed](#) and NASDAQ [Market Data Feeds](#), which stream trading data as it happens and often provide the lowest latency due to being the original data source. Finally, news feeds, like those from [MT Newswires](#), are a common type of data feed, delivering real-time news headlines and text.

One challenge when dealing with financial data feeds is information overload. This happens when a large volume of data from one or more feeds overwhelms the existing financial data infrastructure. The adoption of cloud technology has alleviated this issue by allowing financial institutions to store and retrieve data at any scale without the need to manage and maintain the underlying infrastructure.²⁰

Secure File Transfer

File transfer is one of the most frequent and regular operations in the financial sector. Banks, for example, transfer files for loan applications, transaction history, and account information; investment firms transfer trade data, portfolio composition, and intellectual property files like investment strategy or trading algorithms; insurance companies transfer client policy and personal information files; and regulated financial institutions must submit various filings to comply with regulatory reporting requirements.

Importantly, files transferred by financial institutions often contain sensitive information. As a result, file sharing needs to be secured to meet the financial sector's security and privacy requirements. To this end, a widely used technology is the *Secure File Transfer Protocol* (SFTP), which leverages the SSH protocol to encrypt data and commands that one machine submits to another. SFTP is secure, reliable, and platform independent.

File transfer via SFTP is a good solution for bulk and large file transfers. It is also often used when there isn't an API available for data exchange. Crucially, SFTP may not be the best option for all use cases. For example, it may not be ideal in high-speed and large-volume data-driven systems characterized by demanding workloads. Furthermore, SFTP may require a security policy to manage passwords, keys, and user access, which can increase the complexity of file transfer.

Various alternatives have been proposed to address these issues. For instance, managed SFTP solutions, like [Managed File Transfer \(MFT\)](#), provide enhanced enterprise-level functionality for security, performance, compliance, and reporting beyond what standard SFTP offers. Additionally, protocols such as FTPS (a more secure form of FTP), SCP (Secure Copy Protocol), and WebDAV (Web Distributed Authoring and Versioning) can be utilized depending on the specific business requirements.

Cloud Access

With the widespread adoption of cloud computing across nearly every sector, cloud-based data sharing and access has emerged as a reliable and

convenient way to exchange data. In its simplest setup, a data provider creates a storage bucket or cloud database within a dedicated and isolated cloud environment. It then uploads the data and authorizes the target user to access and manipulate it. Subsequently, whenever new data updates are available, the provider uploads them to the storage location, enabling immediate access for the user.

One convenient aspect of this access method is that users can leverage various cloud-based features when working with the data, including user interfaces, querying capabilities, data management, search functions, and more. Furthermore, there are cost-saving benefits and seamless integration experience with other cloud services, particularly for clients already utilizing the cloud. For instance, if your data pipelines are already processed in the cloud, having a new data source accessible directly through the cloud simplifies workflow management. Additionally, data updates occur continuously with minimal intervention required on the user's part.

Here are a few examples. In 2022, Bloomberg and Google announced a new [partnership](#) that will allow mutual customers to easily access B-PIPE, Bloomberg's real-time market data feed, through Google Cloud. Similarly, CME Group, the world's leading derivatives marketplace, [partnered with Google Cloud](#) to provide fast and reliable market data access to their customers.

CASE STUDY: FACTSET INTEGRATION WITH AWS REDSHIFT AND SNOWFLAKE

FactSet is a well-known financial data vendor that provides content to more than 160,000 investment professionals worldwide. FactSet has expanded its data delivery options in the past few years to include cloud-based access. For example, it is now possible to access 100+ FactSet proprietary and third-party financial datasets through popular cloud data warehouse services such as [AWS Redshift](#) and [Snowflake](#).

With this cloud-based delivery, FactSet saves its clients the need to clean, model, and normalize the data. It is already populated in SQL tables and is ready for users to query. Data is constantly updated and added to existing tables. In addition, if the user relies on both FactSet proprietary data and third-party vendor data offered via FactSet, cloud delivery allows centralization and integration of disparate data sources into a single platform such as Redshift or Snowflake.

Cloud providers have recently increased their market competition in the financial data market. This is expressed by the emergence of the so-called financial data marketplaces. These managed cloud solutions allow financial data providers to distribute and share their data through a single cloud interface. This may be convenient for financial data providers as it eliminates the need to build and maintain an infrastructure for data storage, distribution, billing, and user management. Examples include [AWS Data Exchange for Financial Services](#) and [Google Cloud Datashare for financial services](#). Cloud marketplace data can be delivered in a variety of ways. For example, Google Datashare distributes data in batches through its managed warehouse solution BigQuery and in real time through its managed messaging service Pub/Sub.

Web Access

A user-friendly and straightforward way to access financial data is through a dedicated web page provided by a financial institution or data vendor. In this web access mode, financial data can be downloaded in file format, queried and visualized using a query builder, or quickly parsed and analyzed by the user.

This mode is convenient for handling small datasets or when speed is not critical, such as for scheduled data extractions. However, for faster data access or bulk file downloads, alternative data ingestion technologies like SFTP, API, or cloud-based solutions are more suitable.

Specialized Financial Software

In certain financial data exchange settings, specialized or dedicated software is implemented. This is particularly common when setting up standardized systems for secure financial messaging, payments, transactions, and other market operations.

An example of specialized financial software is [FIX engines](#), which are software applications that enable two institutions to exchange FIX messages (discussed in the previous section). The FIX engine handles the network connection, transmits and receives FIX messages, and validates the submitted messages against the FIX protocol and format.

Data Ingestion Best Practices

When building a data ingestion layer, it is important to make sure it is rock solid. A poorly designed data ingestion layer may easily become the bottleneck of your infrastructure and impact the entire financial data engineering lifecycle. Adhering to best practices can help ensure the resilience of this layer. This section discusses a few.

Meet Business Requirements

First of all, the data ingestion layer needs to meet the business requirements established by your financial institution. Don't overcomplicate the ingestion layer; if your organization wants to work and exchange CSV and Excel files, then build a simple ingestion layer that can process these formats. If an API is not going to bring great benefits, don't build one and instead use a simple filesystem or cloud storage solution. However, if your company has complex data ingestion needs, consider building an extensible and flexible layer that can handle new types of formats and ingestion mechanisms.

Design for Change

Change is a constant in financial markets. New practices, standards, and regulations are continually taking place across the industry. For example, it is not unusual for markets to begin migrating to a newly published standard, and, in the meantime, a new standard emerges to replace it.²¹ For this reason, always consider the change dynamics that might affect your data ingestion layer. There isn't a fixed recipe for managing change, but you can consider best practices such as the following:

Incremental change

Make small and gradual changes.

Isolated change

Make sure you develop and test your changes in isolation and avoid release incompatibility.

Documented change

Make sure you describe the what, how, and why of your changes so others can understand what will change.

Zero downtime

Roll out changes using a reliable technique with a rollback option to ensure zero downtime and avoid disruption to end users.

Examples include Blue/Green, Canary, and Rolling deployment techniques.²²

Enforce Data Governance

Enforcing data governance at the ingestion layer is recommended in financial applications. Examples of good practices include the following:

Data validation

Add validators to check the conformity of ingested data to defined acceptance criteria. For example, you can validate if a file format is CSV or XML, validate an ingested message against standard requirements (e.g., FIX or XBRL), or validate ingested data against errors and quality issues.

Logging and reporting

Consider having an audit log that records all ingestion events. This is useful for tracking wrongful or malicious ingestions and for regulatory reporting.

Lineage and visibility

Implement a mechanism that allows ingested data to be traced back to its origin. The goal is to determine when, how, and where a particular piece of data entered the system.

Security

The data ingestion layer is the entry point for your financial data infrastructure. This means that it can be exploited by cybercriminals to ingest malicious data or software. To this end, ensuring the security of your data ingestion needs to be a top priority. Malicious data can be ingested in different ways, such as malware files, SQL ingestion, and data poisoning.²³ Security can be ensured via proper authentication and authorization policies, user permission management, virus scanning, allowed file formats (e.g., you may want to discard zip or pickled files), API security, and more.

Perform Benchmarking and Stress Testing

A good practice when building a data ingestion layer is performing a stress test to check the infrastructure's ability to handle variable workloads and ingestion scenarios. This is particularly essential when designing an event-driven or real-time data ingestion layer. A useful testing technique is benchmarking, which can be useful when choosing between different ingestion technologies. A benchmarking tool can be used for assessing the performance of a given ingestion technology by simulating a realistic workload scenario.²⁴

Summary

In this chapter, you explored the first layer of the FDEL: ingestion. This layer acts as the entry point to a financial data infrastructure, enabling data to be ingested through various arrival processes, transmission protocols, formats, and technologies. This chapter covered various aspects of ingestion specific to financial data, focusing on the unique requirements and preferences of market participants.

The ingestion layer should be regarded as a vital component of your FDEL. Bottlenecks at this stage can lead to performance issues further downstream. Additionally, a robust ingestion system is becoming increasingly essential for financial institutions due to the growing trends in data sharing, the expanding variety of data formats and arrival processes, and the increasing volumes of ingested data.

At this point, you may be asking yourself, “Where does the ingested data go?” This takes us to the next layer, storage, which will be covered in [Chapter 8](#). In this layer, a data storage system is used to store and retrieve data according to specific business and technical requirements. As you might have guessed, this is the layer where you implement the most popular technology in data engineering: a database.

Let's continue!

- ¹ For more on this topic, I recommend reading this blog post by Google Cloud: [“Authorize with SSL/TLS Certificates”](#).
- ² To learn more about VPCs and subnets, I recommend checking the documentation of AWS on [IP addressing for your VPCs and subnets](#).
- ³ Systems that can handle duplicate ingestions reliably are often called *idempotent*. To learn more about this topic, I recommend the article [“Idempotency and Ordering in Event-Driven Systems”](#), by Wade Waldron.
- ⁴ For a detailed study on this topic, I recommend Sunila Gollapudi's [“Aggregating Financial Services Data Without Assumptions: A Semantic Data Reference Architecture”](#), in the *Proceedings of the 2015 IEEE 9th International Conference on Semantic Computing* (IEEE, 2015): 312–315.
- ⁵ The full guide is available on Snowflake's [website](#) and it assumes you have satisfied all the necessary requisites such as granting roles and permissions between Snowflake and AWS S3, creating the Snowflake table, and other tasks.

- ⁶ To upload files into a stage location, you need to use the Snowflake [PUT command](#).
- ⁷ XML's popularity has remarkably increased in recent years. To read more about this topic, see [“Making Life Easier in an XML World”](#), by Denise Warzel.
- ⁸ For a comparison of Avro and Parquet formats, see [“AVRO vs. PARQUET” by Snowflake](#).
- ⁹ See, for example, Aldane Haldane, Robleh D. Ali and Paul Nahai-Williamson's [“Towards a Common Financial Language”](#), presented at the Securities Industry and Financial Markets Association Symposium on “Building a Global Legal Entity Identifier Framework,” New York, 2012.
- ¹⁰ The sample message was taken from the [“Sample Messages Document”](#), by NYSE. A dictionary of all FIX tags and their meaning is [available online](#).
- ¹¹ Detailed technical specifications on FIX networks and engines are available at the [FIX community website](#).
- ¹² For more details, check the [official XBRL Essentials guide](#).
- ¹³ The source of this example is the [official FpML Coding Schemes documentation](#).
- ¹⁴ To see the full message body and have a more detailed idea about the meaning of the tags, download the full message data [online](#).
- ¹⁵ To learn more about the ISO 20022 registration process, see the official web page on the [development of new ISO 20022 message definitions](#).
- ¹⁶ For a detailed analysis, I recommend checking the implications of ISO 20022 on the payment industry. For this reason, I highly recommend the paper by Steve Goswell, [“ISO 20022: The Implications for Payments Processing and Requirements for Its Successful Use”](#), *Journal of Payments Strategy & Systems* 1, no. 1 (Autumn 2006): 42–50.
- ¹⁷ To learn more about these and other API optimization topics, I highly recommend the book by De Brajesh, *API Management: An Architect's Guide to Developing and Managing APIs for Your Organization* (APress, 2017).
- ¹⁸ For a good reference on API security, see Neil Madden's *API Security in Action* (Manning, 2020).
- ¹⁹ For a detailed analysis of SQL injection, I recommend the seminal paper by William G. Halfond, Jeremy Viegas, and Alessandro Orso, [“A Classification of SQL Injection Attacks and Countermeasures”](#), in the *Proceedings of the International Symposium on Secure Software Engineering*, vol. 1, (March 2006): 13–15.
- ²⁰ For an overview of this issue, see [“The Relentless Rise of Real-Time Data”](#), by LSEG.
- ²¹ For a good reference on this problem, I recommend Chris Pickles' [“Securities Standards Migration: ISO 15022 vs ISO 20022”](#), *Journal of Securities Operations & Custody* 1, no. 3 (Spring 2008): 289–300.
- ²² To learn about these techniques, I suggest Chaitanya K. Rudrabhatla's [“Comparison of Zero Downtime Based Deployment Techniques in Public Cloud Infrastructure”](#), in the *2020 Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)* (IEEE, 2020): 1082–1086.
- ²³ Data poisoning is a type of attack where data is intentionally ingested to alter the performance or behavior of a machine learning model. For more on this topic, I recommend Antonio Emanuele Cinà, Kathrin Grosse, Ambra Demontis,

Sebastiano Vascon, Werner Zellinger, Bernhard A. Moser, Alina Oprea, Battista Biggio, Marcello Pelillo, and Fabio Roli's [“Wild Patterns Reloaded: A Survey of Machine Learning Security Against Training Data Poisoning”](#), *ACM Computing Surveys* 55, no. 13s (July 2023): 1–39.

- ²⁴ For a reference on benchmarking financial data ingestion, I recommend Manuel Coenen, Christoph Wagner, Alexander Echler, and Sebastian Frischbier's [“Benchmarking Financial Data Feed Systems”](#), in the *Proceedings of the 13th ACM International Conference on Distributed and Event-based Systems* (June 2019): 252–253.